



Manual técnico de instalación y configuración

Sistema: Tiendita Maday

Cliente: La Familia

Versión: 2.0

Fecha: 16 de julio de 2026

Revisión base: a9799be

Guía paso a paso para preparar el entorno, instalar frontend/backend/base, configurar servicios y validar una ejecución local o en Railway. Los ejemplos usan marcadores: nunca copie secretos reales a este documento.

1. Alcance del manual

Este manual cubre:

- Requisitos del equipo técnico.
- Clonado y estructura del repositorio.
- Instalación local directa.
- Instalación local con Docker Compose.
- Variables del backend y frontend.
- Inicialización y migraciones PostgreSQL.
- Configuración de Google, PayPal, Cloudinary y correo.
- Despliegue de frontend/backend/base en Railway.
- Pruebas de humo, actualización y diagnóstico.

El respaldo, restauración y atención de incidentes se amplían en `02_Guia_de_Operacion_Respaldo_y_Recuperacion.pdf`.

2. Requisitos

Herramienta	Recomendación	Verificación
Git	Versión vigente	<code>git --version</code>
Node.js	24 LTS; misma versión en desarrollo/CI	<code>node --version</code>
npm	Incluido con Node	<code>npm --version</code>
PostgreSQL	Cliente y servidor compatible	<code>psql --version</code>
Docker Desktop/Engine	Opcional para Compose	<code>docker version</code>
Navegador	Chrome/Edge/Firefox/Safari vigente	Abrir herramientas de desarrollo

Angular 21.2 requiere una versión moderna de Node. No use una versión impar no LTS. En contenedores y CI fije versión/parche o digest en lugar de `latest`.

2.1 Puertos locales

Servicio	Puerto esperado
Frontend Angular	4200
Backend Express	3000
PostgreSQL directo	5432
PostgreSQL por Compose (host)	5433

Compruebe que no estén ocupados o cambie el mapeo antes de iniciar.

3. Obtener el código

1. Abra una terminal en la carpeta de trabajo.

2. Clone el repositorio:

```
git clone https://github.com/CHidalgo405/web_integral.git
```

3. Entre a la carpeta clonada.

4. Confirme rama y revisión:

```
git switch main
```

```
git pull --ff-only origin main
```

```
git rev-parse --short HEAD
```

5. No modifique ni ejecute archivos sin revisar el estado:

```
git status --short
```

4. Ruta A: instalación local directa

Use esta ruta cuando desea depurar frontend/backend por separado y ya tiene PostgreSQL.

4.1 Crear una base vacía

1. Cree un usuario y base exclusivos para el entorno.

2. Evite contraseñas de ejemplo.

3. Desde la raíz del repositorio ejecute, sustituyendo los marcadores:

```
createdb --host=localhost --port=5432 --username=<DB_ADMIN> tiendita_maday
```

```
psql --host=localhost --port=5432 --username=<DB_ADMIN> --dbname=tiendita_maday  
--file="mi cliente (la familia)/Arquitectura/Database/init.sql"
```

Use `init.sql` únicamente sobre una base nueva. En una instalación existente aplique migraciones, no reinicialice.

4.2 Configurar backend

1. Entre a `mi cliente (la familia)/Arquitectura/Backend`.

2. Instale exactamente el lockfile:

```
npm ci
```

3. Cree un archivo local `.env` no versionado con nombres equivalentes:

Variable	Ejemplo no sensible / regla
DATABASE_URL	postgresql://USUARIO:CLAVE@localhost:5432/tiendita_maday
PORT	3000
FRONTEND_URL	http://localhost:4200
BACKEND_URL	http://localhost:3000/api
JWT_SECRET	Aleatorio, largo y distinto por entorno.

Variable	Ejemplo no sensible / regla
JWT_REFRESH_SECRET	Aleatorio y diferente al anterior.
ACCESS_TOKEN_EXPIRATION	2h u otro valor acordado.
GOOGLE_CLIENT_ID	Identificador público del cliente OAuth.
PAYPAL_API_BASE	Sandbox por defecto.
PAYPAL_CLIENT_ID / PAYPAL_SECRET	Credenciales sandbox; nunca en Git.
PAYPAL_CURRENCY	MXN
CLOUDINARY_URL	URL secreta del entorno.
EMAILJS_*	Claves, servicio y plantillas del proveedor activo.

Genere secretos con una fuente criptográfica, por ejemplo:

```
openssl rand -hex 32
```

No copie la salida en tickets, capturas o historial compartido.

4.3 Aplicar migraciones y arrancar backend

Desde la carpeta Backend:

```
npm run migrate
```

```
npm run dev
```

Compruebe:

- `http://localhost:3000/` devuelve el mensaje de API activa.
- `http://localhost:3000/api-docs` abre Swagger UI.
- `http://localhost:3000/swagger.json` devuelve JSON.

Si `npm start` se usa en lugar de `npm run dev`, aplica migraciones antes de iniciar.

4.4 Configurar y arrancar frontend

1. Abra otra terminal.
2. Entre a mi cliente (la familia)/Arquitectura/FrontEnd.
3. Instale dependencias:

```
npm ci
```

4. Confirme que `src/environments/environment.ts` usa `http://localhost:3000/api`.

5. Ejecute:

```
npm start
```

6. Abra `http://localhost:4200`.

El `GOOGLE_CLIENT_ID` del frontend es público por diseño, pero debe coincidir con el backend y tener los orígenes correctos en Google.

5. Ruta B: Docker Compose

Esta ruta crea frontend, backend y PostgreSQL en una red local.

5.1 Preparar variables

En la raíz, cree un `.env` local con al menos:

- `POSTGRES_USER`: usuario no predeterminado cuando sea posible.
- `POSTGRES_PASSWORD`: contraseña fuerte, no `yourpassword`.
- `POSTGRES_DB`: `tiendita_maday` o nombre acordado.

Agregue al servicio backend las variables de JWT y terceros necesarias. El Compose actual declara la conexión de base, `PORT` y `MIGRATIONS_DIR`; revise el archivo antes de usar integraciones.

5.2 Construir y ejecutar

```
docker compose build --pull
```

```
docker compose up
```

En otra terminal:

```
docker compose ps
```

```
docker compose logs --tail=100 backend
```

La primera creación del volumen ejecuta `init.sql`. Volver a levantar el mismo volumen no reinicializa la base.

5.3 Detener

```
docker compose down
```

No use `docker compose down -v` salvo que desee eliminar deliberadamente todos los datos locales y tenga respaldo.

6. Acceso inicial seguro

`init.sql` contiene datos de demostración y hashes de cuentas iniciales. No deben considerarse credenciales de producción.

Procedimiento:

1. Mantenga la instalación en una red controlada.
2. Defina quién será propietario administrador.
3. Cree o restablezca una cuenta mediante un procedimiento técnico controlado.
4. Use una contraseña temporal única y marque cambio obligatorio.
5. Inicie sesión, cambie la contraseña y desactive cuentas demo.
6. Revise roles, correos y empleados vinculados.
7. Registre la entrega de la cuenta sin escribir la contraseña.

No publique en manuales la contraseña correspondiente a hashes incluidos en datos demo. Si no se puede crear el primer administrador sin editar datos, documente y ejecute un script de bootstrap de una sola vez, revíselo y elimínelo

del entorno después.

7. Configurar integraciones

7.1 Google

1. Cree/seleccione un cliente OAuth web.
2. Registre `http://localhost:4200` para desarrollo y el dominio final.
3. Use el mismo `GOOGLE_CLIENT_ID` en frontend y backend.
4. Pruebe alta e inicio de sesión con una cuenta no privilegiada.
5. Confirme que Google no asigna por sí mismo un rol administrativo.

7.2 PayPal

1. Cree aplicación sandbox.
2. Configure client ID/secret solo en backend.
3. Mantenga `PAYPAL_API_BASE` en sandbox.
4. Defina moneda MXN.
5. Pruebe aprobación, cancelación, error y doble clic.
6. Pase a live solo con aprobación y credenciales separadas.

7.3 Cloudinary

1. Cree entorno/carpeta para productos.
2. Configure `CLOUDINARY_URL` en backend.
3. Pruebe carga, lectura, reemplazo y eliminación.
4. Limite formatos/tamaño y revise que no se exponga el secreto.

7.4 Correo

1. Confirme qué proveedor será el único activo.
2. Configure servicio, plantillas de verificación y recuperación.
3. Use dominios/remitentes autorizados.
4. Pruebe entrega, spam, expiración, reintento y código inválido.
5. No registre OTP ni enlaces completos en logs de producción.

8. Compilar y probar

8.1 Frontend

Desde FrontEnd:

```
npm run build
```

Resultado esperado: código 0 y bundle en `dist/la-familia/browser`. Las advertencias de fuentes/CommonJS deben revisarse, aunque no detengan el build.

El comando `npm test` actualmente puede terminar con “No test files found”. No lo marque aprobado hasta corregir la configuración y ejecutar casos reales.

8.2 Backend

Desde Backend:

`npm test`

Resultado de referencia: 19 aprobadas, 0 fallidas y 2 omitidas cuando faltan bases aisladas.

Para las integrales, cree bases desechables y configure:

- `POS_TEST_DATABASE_URL`
- `INVENTORY_TEST_DATABASE_URL`

Nunca apunte estas variables a producción.

8.3 Prueba de humo

- [] API raíz y Swagger responden.
- [] Frontend carga en ventana privada.
- [] Login correcto e incorrecto muestran resultados adecuados.
- [] Cada rol llega a su espacio y no a otro.
- [] Catálogo consulta datos.
- [] Cajero abre caja y registra una venta de prueba.
- [] Almacén registra un ajuste con auditoría.
- [] Administrador consulta productos/usuarios.
- [] Logs no muestran secretos ni errores repetitivos.

9. Despliegue en Railway

9.1 Crear proyecto y PostgreSQL

1. Conecte el repositorio al proyecto Railway.
2. Agregue PostgreSQL.
3. Inicialice una base nueva con `Database/init.sql` mediante consola o `psql`.
4. Registre propietario, plan, región, URL privada y política de respaldo.

9.2 Backend

Existen dos caminos versionados:

Camino	Configuración
Subcarpeta	Root Directory <code>mi_cliente (la familia)/Arquitectura/Backend</code> y su <code>railway.json</code> .

Camino	Configuración
Raíz	Root vacío + Dockerfile personalizado <code>Dockerfile.backend.railway</code> .

Elija uno, documente cuál y no mezcle sus Dockerfiles.

Variables mínimas: `DATABASE_URL`, secretos `JWT`, `FRONTEND_URL`, `BACKEND_URL` y las integraciones habilitadas. Railway inyecta `PORT`.

Genere dominio y valide `/`, `/api-docs` y logs de migración. Si el camino de raíz no incluye migraciones, configúrelas explícitamente antes de exponer tráfico.

9.3 Frontend

Camino	Configuración
Subcarpeta	Root Directory <code>mi_cliente (la familia)/Arquitectura/FrontEnd</code> .
Raíz	Root vacío + <code>Dockerfile.frontend.railway</code> .

Defina `API_BASE_URL=https://<backend>/api` y, si aplica, `GOOGLE_CLIENT_ID`. Como la URL se compila dentro del bundle, cualquier cambio exige reconstruir el frontend.

9.4 Cerrar CORS

Después de obtener el dominio del frontend, configure `FRONTEND_URL` en backend con ese origen exacto y vuelva a desplegar. Verifique que un origen diferente sea rechazado.

10. Actualización controlada

1. Sincronice y registre el commit objetivo.
2. Lea migraciones y variables nuevas.
3. Cree respaldo de base.
4. Ejecute tests y build en esa revisión.
5. Despliegue backend/migraciones compatibles.
6. Compruebe logs, raíz y OpenAPI.
7. Despliegue frontend con URL correcta.
8. Ejecute prueba de humo.
9. Registre hora, responsable y resultado.
10. Si falla, aplique rollback solo después de evaluar compatibilidad de datos.

11. Diagnóstico

Síntoma	Comprobación	Acción
Backend no inicia	DATABASE_URL, logs, puerto	Corregir conexión; no omitir migraciones.
"Migration failed"	Archivo/tabla schema_migrations	Restaurar copia de prueba, revisar SQL y orden.
Frontend llama localhost en producción	Bundle/variable API_BASE_URL	Corregir variable y reconstruir.
Error CORS	FRONTEND_URL exacta	Quitar barra/variación incorrecta y redespargar.
Login siempre falla	cuenta activa, hash, JWT, conexión	Restablecer cuenta por método seguro.
Google rechaza	client ID y orígenes	Alinear frontend/backend/consola Google.
PayPal falla	sandbox/live, moneda y credenciales	No mezclar ambientes; revisar respuesta.
Imagen no sube	CLOUDINARY_URL, tamaño/formato	Revisar configuración y límites.
Correo no llega	proveedor, plantilla, spam	Validar remitente y credenciales.
Tests integrales omitidos	variables de bases test	Crear bases desechables y reejecutar.

12. Entrega técnica

- [] URLs de frontend/API verificadas.
- [] Commit o etiqueta registrado.
- [] Cliente controla GitHub, Railway, base y terceros.
- [] Secretos temporales rotados.
- [] Cuenta propietaria validada y demos desactivadas.
- [] Build y pruebas anexados.
- [] Respaldo exportable y restauración probada.
- [] Monitoreo y responsables definidos.
- [] Pendientes anotados en el acta.

Una instalación "funcionando" no está lista para producción mientras conserve credenciales demo, URLs abiertas, dependencias latest, respaldo no probado o una integración sin sandbox.